

day 7

Markdown

```
# Node.js Prerequisites: Advanced JavaScript (ES6+ Concepts)
```

```
---
```

```
## 📖 1. Let, Const, Var & The Temporal Dead Zone (TDZ)
```

الأولية، وخصوصاً المتعلقة `var` لحل مشاكل `const` و `let` تم تقديم ES6، في إصدار الـ Hoisting والنطاقات.

```
### TDZ والـ `const` و `let` مع Hoisting أ) الـ
```

للمتغيرات المعرفة بـ Hoisting سؤال إنترفيو شهير: هل يتم عمل [!IMPORTANT] `const` و `let`؟

لهما، ولكن يتم حجزهما في منطقة مشفرة في Hoisting الإجابة: **نعم، يتم عمل** > أو (منطقة الموت المؤقتة). لا يمكن **Temporal Dead Zone (TDZ)** الذاكرة تسمى الوصول للمتغير وهو داخل هذه المنطقة حتى يمر المتصفح على سطر التعريف الفعلي (التهيئة المبدئية) Initialization وتتم عملية الـ

```
```javascript
// console.log(x);
// ❌ النتيجة: Uncaught ReferenceError: Cannot access 'x' before
initialization
let x = 10;
```

- تفكيك اللغز: المحرك علم بوجود x (لأنه عمل Hoisting)، لكنه رفض قراءتها لأنها في الـ TDZ.
- الفرق بينها وبين متغير غير موجود نهائياً:

## JavaScript

```
// console.log(notDefinedVar);
// ❌ النتيجة: Uncaught ReferenceError: notDefinedVar is not defined
```

## ب) الارتباط بالكائن العالمي (Global Window Object Binding)

- المتغيرات المعرفة بـ var في النطاق العالمي يتم تخزينها كـ Property مباشرة داخل كائن الـ window (في المتصفح).
- المتغيرات المعرفة بـ let و const لا تُربط بكائن الـ window بل تُخزن في نطاق خاص بها (Script/Declarative scope).

## JavaScript

```

var globalVar = "Iam global var";
console.log(globalVar); // "Iam global var"
console.log(window.globalVar); // "Iam global var"

let globalLet = "Iam global let";
console.log(globalLet); // "Iam global let"
console.log(window.globalLet); // ⚠ undefined (لأن let الـ لا تُسجل في الـ window)

const globalConst = "Iam global const";
console.log(globalConst); // "Iam global const"
console.log(window.globalConst); // ⚠ undefined

```

## 🌐 2. Scopes Deep Dive (تحليل النطاقات الشامل)

### أ) نطاق الدالة (Function Scope)

جميع الكلمات المفتاحية ( `var` , `let` , `const` ) يتم حبسها وتقييدها داخل نطاق الدالة إذا عُرِفَتْ بداخلها، ولا يمكن تسريبها للخارج.

JavaScript

```

function sayHello() {
 var x = 10;
 let y = 20;
 const z = 30;
}
sayHello();

// محاولة الوصول لهم خارج الدالة تفشل تماماً:
// console.log(x); // ❌ ReferenceError: x is not defined
// console.log(y); // ❌ ReferenceError: y is not defined
// console.log(z); // ❌ ReferenceError: z is not defined

```

### ب) نطاق الأقواس (Block Scope)

الـ Block هو أي كود محصور بين أقواس مجمدة {} مثل جمل `if` , `for` , `while` .

- `var` : الـ Global وتختبره لتصبح **Block Scope** لا تحترم الـ Block.
- `let` و `const` : **Block Scope** تحترم الـ Block وتُحبس بداخله فوراً.

JavaScript

```

if (true) {
 var blockVar = 10;
}

```

```

let blockLet = 20;
const blockConst = 30;
}

console.log(blockVar); // 10 (وخرج بنجاح if اخترق الـ)
// console.log(blockLet); // ❌ ReferenceError: blockLet is not defined
// console.log(blockConst); // ❌ ReferenceError: blockConst is not defined

```

### 3. Re-declaration vs Re-assignment (إعادة التعريف والتعيين)

الكلمة المفتاحية	إعادة التعريف في نفس النطاق (Redeclare)	إعادة تعيين القيمة (Reassign)
var	✓ مسموح	✓ مسموح
let	❌ ممنوع	✓ مسموح
const	❌ ممنوع	❌ ممنوع

#### JavaScript

```

// 1. مع var: مسموح بكل شيء
var x = 10;
var x = "hello";
console.log(x); // "hello"

// 2. مع let: ممنوع إعادة التعريف، مسموح بتغيير القيمة
let y = 10;
// let y = 20; // ❌ SyntaxError: Identifier 'y' has already been declared
y = "hello"; // ✓ تعديل القيمة مسموح
console.log(y); // "hello"

// 3. مع const: ممنوع التعديل أو إعادة التعريف (ثابت صارم)
const z = 10;
// const z = 20; // ❌ SyntaxError
// z = 15; // ❌ TypeError: Assignment to constant variable
console.log(z); // 10

```

### 4. Modern ES6 Features (المميزات الحديثة)

#### أ) النصوص الذكية والـ Template Literals

باستخدام الباك تيك ( ` ` ) بدلاً من علامات الاقتباس العادية، نكتسب ميزتين:

1. كتابة نص على عدة أسطر (Multiline string) دون الحاجة لـ ` \n `.
2. دمج المتغيرات ديناميكياً باستخدام الكود التعبيري `\${variable}` (يُسمى String Interpolation).

## JavaScript

```
const name = "ali";
const age = 26;

const message = `hello ${name} and age          is ${age}`;

console.log(message);
```

### ب) المعاملات الافتراضية (Default Parameters)

تسمح لنا بوضع قيمة احتياطية للبارامتر في حال نسي المستخدم تمريرها أثناء استدعاء الدالة.

## JavaScript

```
function sayHello(name = "guest") {
 console.log(`hello ${name}`);
}

sayHello("ali"); // "hello ali"
sayHello("ahmed"); // "hello ahmed"
sayHello(); // "hello guest" (undefined استخدم القيمة الافتراضية بدلاً من)
```

### ج) الـ Spread Operator ... (معامل النشر والتوزيع)

يقوم بـ "فك" أو "نشر" عناصر الكائنات القابلة للتكرار (مثل المصفوفات والنصوص والنصوص الكائنية) داخل مكان جديد.

⚠ سؤال إنترفيو خطير: ما الفرق بين نسخ المصفوفة بالتعيين مباشرة أو بالـ **Spread Operator** في الميموري؟

#### 1. النسخ بالتعيين المباشر (Copy by Reference / Pointer):

المصفوفتين بيشاروا لنفس العنوان (Pointer) في الذاكرة. لو عدلت في واحدة، الثانية تتأثر فوراً!

## JavaScript

```
let arr6 = [1, 2, 3];
let arr7 = arr6; // في الذاكرة Pointer المصفوفتان تشيران لنفس الـ

arr6.push(6); // تعديل المصفوفة الأولى
console.log(arr7); // [1, 2, 3, 6] (!!المصفوفة الثانية تغيرت تلقائياً)
```

#### 2. النسخ بالـ Spread Operator (Shallow Copy):

يتم إنشاء مصفوفة جديدة تماماً في عنوان منفصل في الذاكرة، ويتم نقل القيم فقط إليها.

## JavaScript

```
let arr4 = [1, 2, 3];
let arr5 = [...arr4]; // القيم Pointer مصفوفة جديدة تماماً بـ

arr4 = [5, 6, 7]; // إعادة تعيين المصفوفة الأولى بالكامل لعنوان جديد
console.log(arr4); // [5, 6, 7]
console.log(arr5); // [1, 2, 3] (لم تتأثر وضلت محتفظة بالقيم القديمة)
```

### 3. دمج المصفوفات وتوزيع النصوص والكائنات:

## JavaScript

```
// دمج مصفوفات
let arr1 = [1, 2, 3, 4, 5];
let arr2 = [...arr1, 6, 7, 8, 9];
console.log(arr2); // [1, 2, 3, 4, 5, 6, 7, 8, 9]

// تفكيك نص إلى مصفوفة حروف
let str = "hello";
let charArray = [...str];
console.log(charArray); // ['h', 'e', 'l', 'l', 'o']

// دمج وتعديل كائنات (Objects)
let obj1 = { a: 1, b: 2 };
let obj2 = { ...obj1, b: 3 }; // لتصبح 3 بدلاً من 2 b وعدل قيمة obj1 فك
console.log(obj2); // {a: 1, b: 3}
```

### د) الـ Rest Operator ... (معامل التجميع)

نفس شكل الـ Spread بالظبط ( ... ) ولكن مكانه يبقى جوة بارامترات الدالة. وظيفته عكس الـ Spread تماماً: بيلم الأرقام أو الوسائط المشتتة ويجمعها في مصفوفة حقيقية واحدة.

## JavaScript

```
function sum(...args) { // args المدخلات تحتوي على كل المدخلات
 return args.reduce((acc, cur) => acc + cur, 0);
}

console.log(sum()); // 0
console.log(sum(5)); // 5
```

```
console.log(sum(5, 5)); // 10
console.log(sum(5, 5, 5)); // 15
```

## هـ) الدوال السهمية (Arrow Functions)

طريقة مختصرة وحديثة لكتابة الدوال:

- لو الدالة بتنفذ سطر واحد وترجع قيمة، نقدر نشيل الأقواس {} ونشيل كلمة return ويتم الإرجاع تلقائياً (Implicit Return).
- لو الدالة فيها كود معقد، بنحط الأقواس ونكتب كلمة return صراحة.

JavaScript

```
// سطر واحد (Implicit Return)
const sumArrow = (a, b) => a + b;
console.log(sumArrow(5, 5)); // 10

// عدة أسطر (Explicit Return)
let calc = (a, b) => {
 const sum = a + b;
 const diff = a - b;
 return { sum, diff }; // إرجاع كائن يحتوي على النتيجتين
};
console.log(calc(5, 5)); // {sum: 10, diff: 0}
```



## 5. High-Order Array Methods & Native Implementation

الجزء الأهم في الإنترنت فيوهات: فهم واستخدام وتطوير ميثودز المصفوفات الشهيرة من الصفر (Native/Vanilla JS).

### 1. دالة الـ map()

- **وظيفتها:** تلف على المصفوفة وتنفذ دالة معينة على كل عنصر، وترجع مصفوفة جديدة تماماً بنفس طول المصفوفة الأصلية.

JavaScript

```
const names = ["ali", "ahmed", "mohamed"];
const newNames = names.map((name, i) => `${i} - ${name} salem`);
console.log(newNames); // ['0 - ali salem', '1 - ahmed salem', '2 - mohamed salem']
```

🔧 المحاكاة وبناء الدالة من الصفر (Native map Implementation):

JavaScript

```

var mapNative = function (arr, fn) {
 var newArray = []; // مصفوفة جديدة فارغة للحفاظ على عدم تعديل الأصلية
 for (let i = 0; i < arr.length; i++) {
 // index وباصي لها العنصر الحالي والـ (fn) تنفيذ الدالة الممررة
 newArray.push(fn(arr[i], i));
 }
 return newArray;
};

const NativeNames = mapNative(names, (name, i) => `${i} - ${name} salem`);
console.log(NativeNames); // الجاهزة بالظبط map نفس نتيجة الـ

```

## 2. دالة الـ filter()

- **وظيفتها:** تالف على المصفوفة وتصفّيها؛ ترجع **مصفوفة جديدة** تحتوي فقط على العناصر التي حققت الشرط (أرجعت الدالة معها true).

JavaScript

```

const namesList = ["ali", "ahmed", "mohamed"];
const filteredNames = namesList.filter((name) => name.startsWith("a"));
console.log(filteredNames); // ['ali', 'ahmed'] (a العناصر التي تبدأ بحرف)

```

## المحاكاة وبناء الدالة من الصفر (Native filter Implementation) 🛠️

JavaScript

```

var filterNative = function (arr, fn) {
 var newArr = [];
 for (let i = 0; i < arr.length; i++) {
 // للعنصر الحالي، ضيفه للمصفوفة الجديدة true لو الدالة الممررة أرجعت
 if (fn(arr[i], i)) {
 newArr.push(arr[i]);
 }
 }
 return newArr;
};

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9];
const evenNumbers = filterNative(numbers, (number) => number % 2 === 0);
console.log(evenNumbers); // [2, 4, 6, 8]

```

## 3. دالة الـ reduce()

- **وظيفتها:** تُلَف على المصفوفة لتقوم باختزالها ودمج عناصرها كلها لتنتهي بـ **قيمة واحدة فقط** (سواء رقم، نص، أو كائن). تأخذ دالة بداخلها وسيطين أساسيين: الـ **Accumulator** (المُراكم/المخزن الإجمالي) والـ **Current** (العنصر الحالي الحركي)، بالإضافة لقيمة ابتدائية (Initial Value).

## JavaScript

```
const nums = [1, 2, 3, 4, 5];
const totalSum = nums.reduce((accumulator, current) => accumulator + current, 0);
console.log(totalSum); // 15
```

## جدول تتبع عمل الـ reduce خطوة بخطوة (Tracer Table):

الخطوة (Iteration)	قيمة الـ Accumulator (المخزن الحالي)	قيمة الـ Current (العنصر الحركي)	نتاج العملية (acc + cur)	القيمة الجديدة الذاهبة للخطوة التالية كـ acc
البداية الابتدائية	0	-	-	0
الدورة 1	0	1	0 + 1 = 1	1
الدورة 2	1	2	1 + 2 = 3	3
الدورة 3	3	3	3 + 3 = 6	6
الدورة 4	6	4	6 + 4 = 10	10
الدورة 5	10	5	10 + 5 = 15	15 (النتاج النهائي)

## المحاكاة وبناء الدالة من الصفر (Native reduce Implementation):

## JavaScript

```
var reduceNative = function (arr, fn, initialValue) {
 let value = initialValue; // المخزن الإجمالي يبدأ بالقيمة الابتدائية الممررة

 for (let i = 0; i < arr.length; i++) {
 // تحديث قيمة المخزن الإجمالي بناتج تشغيل الدالة مع تمرير (المخزن، العنصر، الاندكس)
 value = fn(value, arr[i], i);
 }

 return value; // إرجاع القيمة الفردية النهائية المحسوبة
};
```



```
const customSum = reduceNative(nums, (acc, cur) => acc + cur, 0);
console.log(customSum); // 15
```